

Data Types & Variable Scoping

Recap of the Day 3 videos: the seven values `typeof` can show you, and where a variable "lives" — global, function or block scope — plus the real reason we avoid `var`.

The basic data types

```
typeof "Palla"    // "string"
typeof 45         // "number"
typeof true       // "boolean"
typeof undefined  // "undefined"
typeof { id: 1 }   // "object"
typeof null       // "object" <-- famous JS quirk!
```

- **undefined** — a variable that was declared but never given a value.
- **null** — an intentional "no value", set by you on purpose.
- `typeof null` returns "object" — a 30-year-old bug that is now part of the language. Interviewers love this one.

Variable scoping

- **Global scope** — declared outside everything; visible everywhere.
- **Function scope** — declared inside a function; invisible outside it.
- **Block scope** — `let` and `const` live only inside the nearest `{ }`. `var` ignores blocks — that is why we avoid it.

```
if (true) {
  let inside = "block only";
  var leaky  = "escapes the block";
}
// console.log(inside); // ReferenceError
console.log(leaky);      // "escapes the block" (var leaked!)
```

Quick reference

Expression	Result
<code>typeof "text"</code>	"string"
<code>typeof 3.14</code>	"number"
<code>typeof false</code>	"boolean"
<code>typeof undefined</code>	"undefined"
<code>typeof null</code>	"object" (the quirk — remember it)
<code>typeof {}</code>	"object"
<code>typeof []</code>	"object" (arrays are objects too)

Practice exercises

Do these in VS Code, run them with Node, and bring questions to the next live Q&A.

1. Before running anything, predict the typeof result for: "45", 45, true, null, undefined, [1, 2]. Then verify each in Node.

Hint: Watch out: "45" in quotes is a string, and two of these say "object".

2. Create an object student with properties name, batch and score. Print just the name.
3. Write an if-block that declares a variable with let inside it, then try to print that variable after the block. What happens?

Hint: ReferenceError — that is block scope protecting you.

4. Repeat exercise 3 with var instead of let. Compare the behaviour.
5. Declare a variable without a value and print it, then set it to null and print again. Explain the difference in a comment.

Bonus challenge — Write a loop: `for (var i = 0; i < 3; i++)` then print `i` AFTER the loop. Now change `var` to `let`. One version prints 3, the other crashes — write a comment explaining why.